

The Agent Stack - Part 5: Context, Retrieval, and Memory

Why context is assembled, not given



VINOTH GOVINDARAJAN

APR 27, 2026



12



1

Share

A user asks:

“Use the policy we discussed last week, but update it with the new onboarding doc.”

The model does not have “last week” unless the runtime reconstructs it.

It does not have “the new onboarding doc” unless retrieval brings it in.

It does not have the user’s project state unless the system puts that state in front of it.

When the answer is wrong, the failure often looks like a model problem. Sometimes it is. But many failures are simpler: the runtime assembled the wrong working set.

Wrong session. Stale evidence. Missing memory. Too much transcript noise. A tool output treated like durable truth.

That is what this layer owns.

Thesis

Context is not what the model knows.

Context is the bounded working set the runtime assembles for one turn.

Retrieval brings evidence into that turn. Memory brings durable state back into that turn. Session history preserves continuity across turns.

Those are related, but they are not interchangeable.

The model only sees the working set

A useful agent system usually has more information than it can send to the model.

It has a transcript. It has state. It has documents. It has tool results. It may have user preferences, summaries, workflow checkpoints, cached files, artifacts, and policy constraints.

The model does not automatically see any of that.

It sees the request payload.

At the API boundary, this becomes concrete. Conversation state has to be carried forward explicitly or through platform-managed state, and every model call still operates within a context window. OpenAI's conversation state docs describe ways to carry prior response output into later requests, while Gemini's long-context docs frame the context window as the information passed to the model for generation.

That payload is the working set for the turn.

It may include:

- instructions
- the latest user message
- selected session history
- workflow state
- retrieved evidence
- durable memories
- tool definitions
- tool outputs
- artifacts

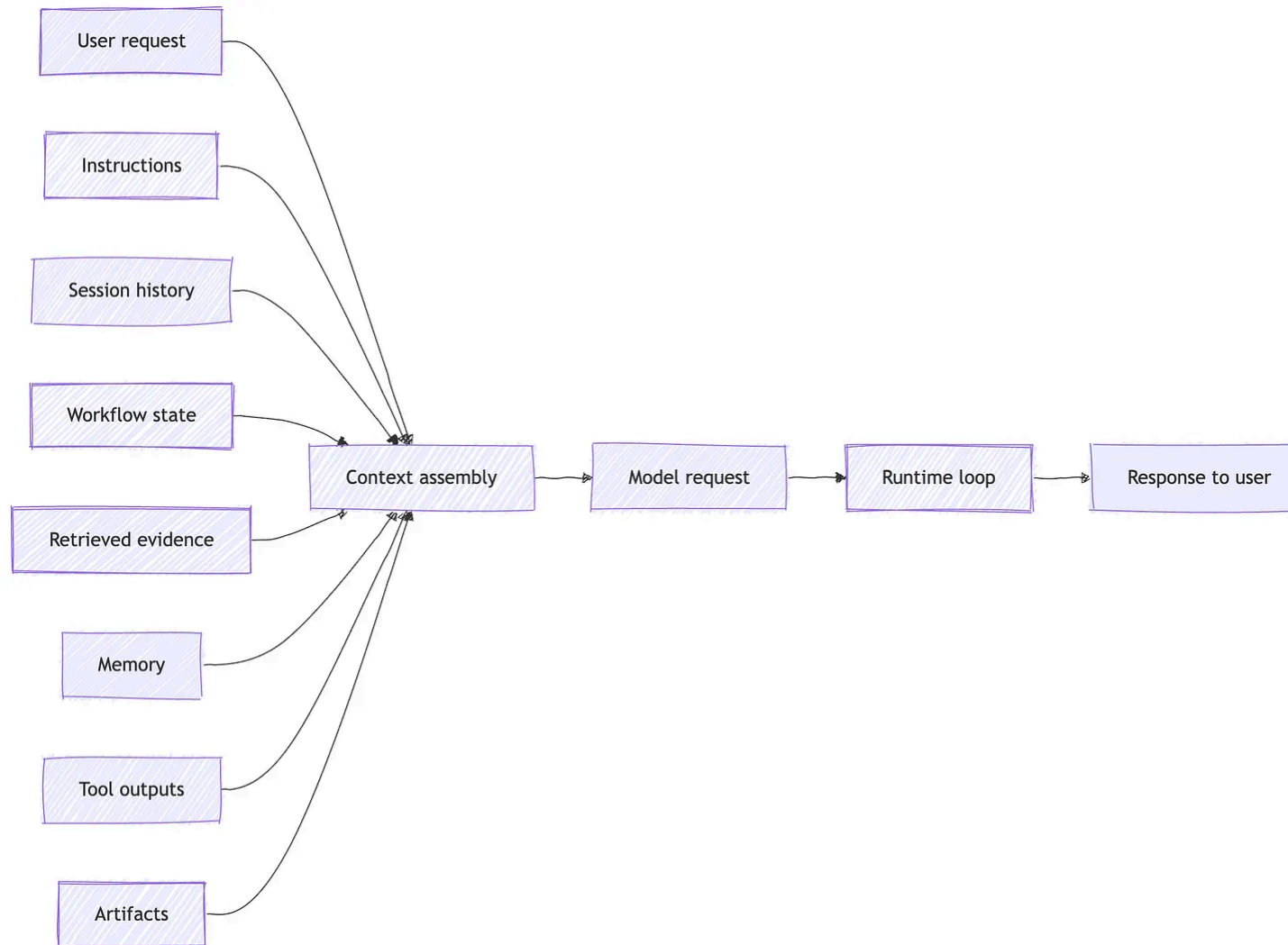
The important word is **selected**.

Context assembly is **the runtime step** that turns all possible inputs into one model-visible request. It filters, ranks, summarizes, formats, and budgets the information the model can reason over.

You can call this prompt construction, but that sounds too small.

The prompt is one artifact. The harder problem is deciding which state and evidence belong in the request at all.

If you remember one picture from this post, make it this one:



That diagram is not a product category.

It is a responsibility boundary.

Something in the system has to answer:

- What does the model need right now?
- What should be omitted?
- What is stale?
- What is scoped to this user?
- What is evidence versus instruction?
- What is safe to trust?
- What can wait until after the response?

If the answer is wrong, you cannot debug only the final text.

You have to inspect the working set.

Context is not what the model knows. It is what the system chooses to put in front of it for this turn.

Session history is source material, not the turn

A session is the unit of continuity.

It is where the system keeps the chronological record of an interaction: user messages, assistant responses, tool calls, tool results, events, and sometimes structured state.

Google's Agent Platform Sessions docs describe sessions as maintaining the history of interactions between a user and agents, including the ability to start, resume, and append new events to a session.

That matters because stored session history and prompt context are not the same thing.

The session history might contain hundreds of events. The model request might include only the last few turns, a compacted summary, a specific tool result, and a small piece of workflow state.

That is fine.

Prompt context is derived state.

Session history is source material.

The system needs to compact aggressively without losing its source of truth. Compaction should mean:

“Do not send all of this to the model right now.”

It should not accidentally mean:

“Erase the only record of what happened.”

That is one way agents become hard to reason about. A user says, “We already talked about this,” and the operator has to untangle several possible failures:

- The wrong session key was used.
- The source transcript exists, but the runtime did not select it.
- A summary dropped the important detail.
- A retrieved memory contradicted the session.
- A tool result was excluded because it looked old.
- The current request exceeded the budget, so the wrong material was trimmed.

Those are different failures.

They need different fixes.

The invariant is simple: do not confuse the stored chronology with the model-visible subset.

The session is the record.

The prompt context is the working set.

Retrieval brings evidence, not authority

Retrieval is how the system brings external evidence into the turn.

That evidence might come from docs, tickets, policies, code, PDFs, file search, search indexes, database rows, or an internal knowledge base.

The standard RAG pipeline matters, but it is not the main point here. Yes, systems ingest data, chunk it, index it, search it, and inject relevant results into a model request. OpenAI's File Search docs describe vector stores that parse, chunk, embed, and store files for keyword and semantic search.

Builders already know that story.

The systems point is this:

Retrieval returns candidate evidence.

It does not return truth.

A retrieved chunk may be relevant and still wrong. It may be semantically similar and still stale. It may be technically accurate but not authorized for this user. It may be

from the right document but the wrong version. It may include untrusted text that should not be allowed to steer the model.

A similarity score means “this looks related.”

It does not mean “this should govern the answer.”

That is why retrieval needs metadata and policy, not just embeddings.

At minimum, retrieved evidence should carry enough information for the runtime, the model, and the operator to reason about it:

- source
- owner or tenant
- document version
- timestamp or freshness
- permissions
- retrieval reason
- confidence or ranking signal
- whether it is user-provided, system-provided, or external

This is an easy place for agent systems to get sloppy.

They build a retrieval path that can find text, but not a context path that can explain why the text belongs in the turn.

That difference matters.

A retrieval layer solves access.

A context layer decides whether accessed evidence should influence the model. | should be in an intelligent way?

Memory is state with a lifecycle

Memory is another overloaded word.

In this series, I mean something narrow:

Memory is durable state persisted outside the model and re-injected into later turns.

It is not model learning.

The model weights are not changing because a user says, “Remember that I prefer concise updates.” In most systems, that means an external store records a preference, and the runtime may later put that preference back into the model context.

Memory is also not just the transcript.

A transcript says what happened.

A memory says what the system chose to preserve from what happened.

That choice is the whole problem.

A memory layer has to decide what survives beyond the current turn. It has to decide who the memory belongs to, how it was created, how long it remains valid, and when it should come back.

LangGraph's memory docs make a similar separation between short-term memory as agent state for multi-turn conversations and long-term memory for user-specific or application-level data across sessions. Google's Memory Bank docs describe long-term memories that persist across sessions, with scoped collections and managed memory lifecycle features such as generation, retrieval, TTL, revisions, and permissions.

The exact product surface varies.

The architectural responsibility is stable.

A memory system has to handle questions like:

- What information is worth extracting?
- Is this explicit user instruction or inferred preference?

- Is it scoped to a user, session, project, tenant, or application?
- What source produced it?
- Does it conflict with an older memory?
- Should the older memory be updated, invalidated, or kept?
- How fresh is it?
- When should it expire?
- Who can inspect or delete it?
- Where should it appear in the next model request?

A vector database can support memory, but it does not own memory lifecycle.

It can help find semantically related records. It cannot, by itself, decide whether a user's old travel preference should still shape a booking recommendation. It cannot decide whether a tool result from three months ago should become durable state. It cannot resolve a conflict between “user prefers short answers” and “user asked for full detail on legal review.”

Those are ownership questions.

My read is that this is the cleanest distinction:

Retrieval solves access. Memory solves ownership.

A memory system may use retrieval. It may use a vector index. It may use structured profiles, summaries, graphs, or plain records.

The architecture question is not “where are the embeddings?”

The architecture question is “who owns the remembered state, and how does it safely return?”

Context assembly is hot-path work

Timing matters.

Context assembly happens before the model can produce the next useful token.

The runtime may need to:

- fetch the session
- load workflow state
- retrieve documents
- retrieve memories
- filter by scope
- trim old history

- attach tool definitions
- include recent tool outputs
- format all of that into the model request

That work is on the hot path.

If it is slow, the user waits.

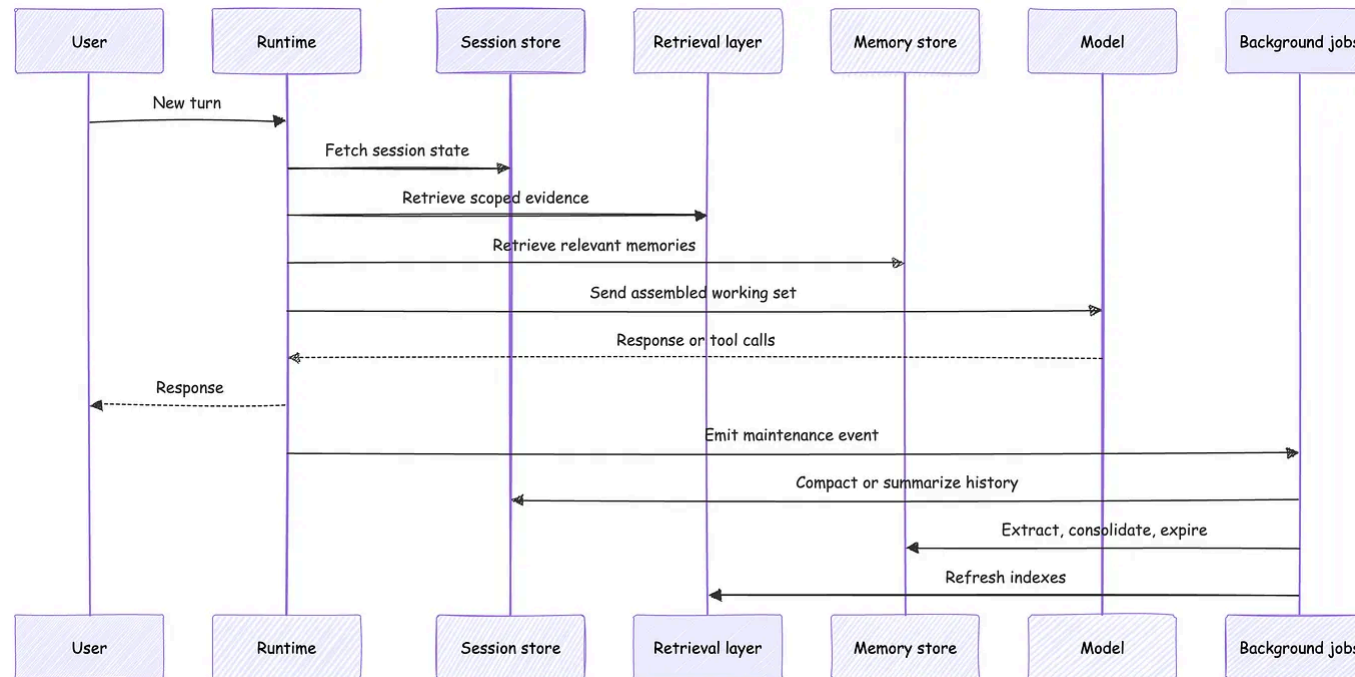
If it is wrong, the model reasons over the wrong material.

Background maintenance is different.

Memory extraction can often happen after the response. So can memory consolidation, index refreshes, transcript compaction, summary generation, cleanup jobs, and stale-state expiration.

Google's Memory Bank docs describe memory generation as a long-running operation and note that **production agents generally run memory generation in the background** because the current run usually does not need the generated memories immediately.

That split changes the design.



The hot path asks:

What does the model need right now?

The background path asks:

What should the system preserve, clean up, update, or forget for later?

If every turn blocks on memory extraction and consolidation, the system pays for it in latency.

Sometimes that is required. If the user explicitly says, “Remember this before we continue,” the write may need to be visible immediately.

But most turns do not need that.

A durable memory can be generated after the user gets a response. The system can emit an event, process it later, dedupe it, retry failures, and consolidate it with existing state.

That is the less glamorous part of memory.

It is not just recall.

It is write policy, timing, retries, and cleanup.

Long context changes the budget, not the responsibility

Long context helps.

It lets the runtime include more history, more evidence, more examples, and more state in one request. Gemini’s long-context docs describe models with context windows of 1 million or more tokens and frame the context window as the information passed to the model for generation.

But long context does not remove the job.

A bigger context window does not know which session belongs to the user. It does not know which policy document superseded the old one. It does not know whether a memory was inferred weakly or stated explicitly. It does not know whether a tool output is safe to persist.

It only gives the system more room.

More room can also mean more noise, more cost, more latency, and more ways to bury the relevant fact. The “Lost in the Middle” paper found that model performance can degrade when relevant information appears in the middle of long inputs, even for explicitly long-context models.

So the question does not go away.

The runtime still has to decide what belongs in the working set.

Prompt caching has a similar boundary.

Caching can make repeated context cheaper or faster. OpenAI describes prompt caching as a context-management feature, Gemini explicit caching lets developers cache input tokens and refer to them in later requests, and Anthropic’s prompt

caching docs describe request blocks such as tool definitions, system messages, text messages, documents, tool use, and tool results as cacheable content.

That is useful.

It is not memory.

Caching does not decide what should be remembered. It does not own deletion. It does not resolve conflicts. It does not validate scope.

A bigger context window changes the budget. It does not decide scope, provenance, or trust.

Why this still leaves tools for the next layer

Context determines what the model can reason over.

Tools determine what the model can ask the system to do.

Those are different boundaries.

A retrieved policy can inform an answer. It does not authorize an action.

A memory can shape personalization. It does not grant identity.

A tool definition can tell the model that a capability exists. It does not decide whether the call should execute against a real browser, code runner, database, payment API, or production system.

OpenAI's tools docs describe tools as a way to extend model capabilities through built-in tools, function calling, tool search, and remote MCP servers. That is exactly why the boundary matters: once context becomes capability, the question shifts from what the model sees to what the system allows it to call.

That is why the next layer still needs separate treatment.

Once the model has the right working set, the next question is not just:

What does it know?

The next question is:

What capabilities has the runtime exposed, under whose authority, and with what controls?

That is where tools, MCP, and capability surfaces enter the stack.

so no tools in the context?

Failure modes

1. Transcript stuffing

The system keeps appending history until the request becomes expensive, slow, noisy, or too large.

This works in demos.

It fails in long-running systems.

2. Compaction as deletion

The runtime trims what the model sees, but accidentally destroys the source material needed for audit, recovery, or later reconstruction.

Compaction should change the working set.

It should not silently rewrite history.

3. Retrieval as truth

The system treats retrieved chunks as authoritative because they ranked highly.

Related text is not the same as trusted evidence.

4. Missing scope

A memory or retrieved document enters the prompt without a clear owner boundary.

This is where personalization becomes leakage.

Every item needs scope: user, tenant, session, project, application, or some explicit combination.

5. Vector database as memory

The team builds a vector index and calls it memory.

Then they discover they still need extraction, conflict handling, provenance, TTL, deletion, and audit.

The index was useful.

It was not the whole layer.

6. Implicit memory writes

The system persists facts because the model said them, a tool returned them, or a user mentioned them once.

Durable memory should be an explicit **lifecycle** event. *So then removed?*

Not every observation deserves to survive.

7. Hot-path memory maintenance

Every turn blocks on extraction, consolidation, and writes.

The agent feels smart in a notebook and sluggish in production.

Move expensive maintenance off the hot path unless the user flow requires immediate persistence.

8. Prompt cache mistaken for memory

A cached prefix makes repeated context cheaper.

It does not create durable state ownership.

Caching optimizes reuse. Memory governs persistence.

9. Memory poisoning

Persistent memory creates a future influence channel.

If malicious or false information gets stored, it can affect later turns after the original input is gone. Recent research has explored memory injection attacks against memory-

enabled agents, and Google's Memory Bank docs explicitly call out memory poisoning as a risk to consider for long-term memories.

This does not mean memory is unsafe by default.

It means memory is a trust boundary.

Builder checklist

If you are building this layer, start with the boundaries.

1. **Name the source of truth for session state.** what a source of truth is?

The transcript, event log, and working state need an owner.

2. **Treat prompt context as derived state.**

The model-visible payload should be inspectable, reproducible, and separate from stored history.

3. **Put scope on every retrieved or remembered item.**

User, tenant, session, project, application. Do not leave this implicit.

4. **Track provenance and freshness.**

The runtime should know where context came from, when it was created, and why it was included.

5. Separate retrieval policy from memory lifecycle.

Retrieval finds candidate evidence. Memory decides what persists and how it changes.

6. Make memory writes explicit.

Do not let every model statement or tool output become durable state.

7. Keep expensive maintenance off the hot path when possible.

Extract, consolidate, compact, expire, and reindex in background jobs unless the current turn needs the result immediately.

8. Audit the assembled working set.

At 3:00 AM, the useful question is not only what the model said. It is what the runtime put in front of it.

Recap

Context is assembled.

It is not the whole database. It is not the whole transcript. It is not everything the system might know.

It is the working set the runtime gives the model for one turn.

Session history preserves continuity.

Retrieval brings external evidence.

Memory brings durable state back into the conversation.

Long context and caching help with budget and performance, but they do not remove the need for selection, scope, provenance, and lifecycle management.

This layer determines what the model can reason over.

The next layer determines what the system lets the model do.

What comes next

Part 6 moves from context to capability.

A tool is not just a function call. It is a boundary where the runtime exposes some piece of the outside world to the model: an API, browser, database, code executor, workflow, local file, remote system, or protocol server.

That changes the question.

Part 5 asked:

What does the model see right now?

Part 6 asks:

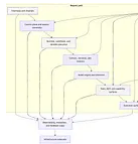
What is the model allowed to call, under whose authority, and with what limits?

That is where tools, MCP, and capability surfaces become their own layer.

Thanks for reading The Agent Stack! Subscribe
for calm, systems-first writing on how agent
infrastructure actually works under the hood.

The Agent Stack v1

New here? Start with [Part 1](#) for the full stack map. Subscribe if you want the rest of the series as it publishes.



The Agent Stack - Part 1: A Systems Map of Modern Agent Infrastructure

VINOTH GOVINDARAJAN • 30 MAR

[Read full story](#) →



The Agent Stack - Part 2: Foundation Infrastructure, Models, and Inference

VINOTH GOVINDARAJAN • 6 ABR

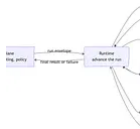
[Read full story →](#)



The Agent Stack - Part 3: Control Planes, Sessions, and State Ownership

VINOTH GOVINDARAJAN • 13 ABR

[Read full story →](#)



The Agent Stack - Part 4: Runtimes, Workflows, and Durable Execution

VINOTH GOVINDARAJAN • 20 ABR

[Read full story →](#)

Sources and further reading

- [OpenAI, Conversation state](#) - for how conversation state is carried across model interactions.
- [OpenAI, File Search](#) - for retrieval over files using chunking, embeddings, and keyword/vector search.
- [Google Agent Platform, Sessions](#) - for session state and event history in agent systems.
- [Google Agent Platform, Memory Bank](#) - for long-term memory, scope, lifecycle, background generation, and memory poisoning risks.

- [LangGraph, Memory](#) - for short-term versus long-term memory and thread-scoped state.
- [Liu et al., Lost in the Middle](#) - for why larger context windows still require careful selection.

Subscribe to The Agent Stack

By Vinoth Govindarajan · Launched 5 months ago

The Agent Stack is a technical deep dive into how modern AI agents actually work under the hood, from control loops and memory systems to orchestration, evaluation, and production deployment.

By subscribing, you agree Substack's [Terms of Use](#), and acknowledge its [Information Collection Notice](#) and [Privacy Policy](#).



12 Likes · 1 Restack

Discussion about this post

Comments

Restacks



Write a comment...